

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Monitoring stack setup — Part 4: Blackbox exporter

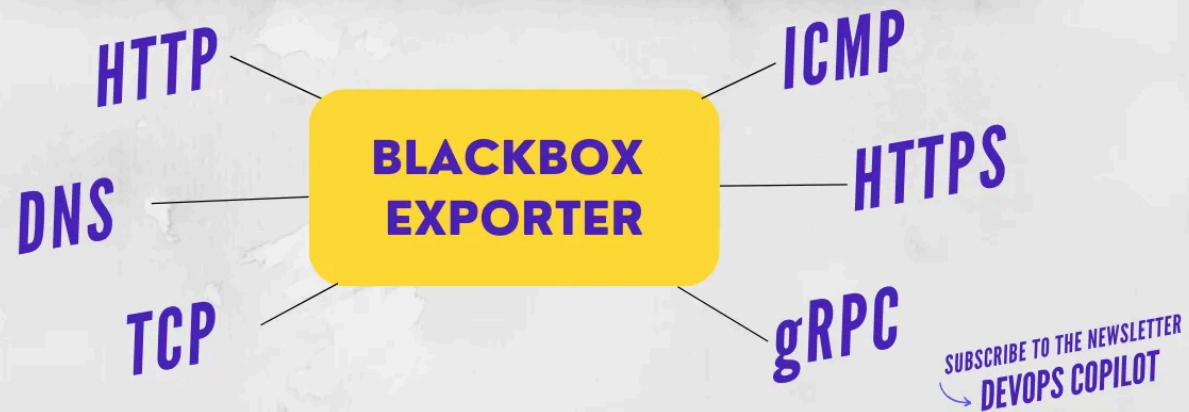


Shishir Khandelwal · [Follow](#)

7 min read · Nov 26, 2022



INSTALLING, CONFIGURING & USING BLACKBOX EXPORTERS



In this article, we are going to see the following —

- Installing blackbox
- Configuring blackbox
- Blackbox concepts
- Sending the metrics to Prometheus for storing metrics.

Blackbox exporters

The blackbox exporter allows the probing of endpoints over HTTP, HTTPS, DNS, TCP, ICMP, and gRPC. It provides metrics about latencies for HTTP requests and DNS as well as statistics about SSL certificate expiration.

The necessity of blackbox exporters

One could be confused as to why such a component exists when developers can add similar monitoring capabilities inside the code using metrics libraries.

The answer is that -

- Those libraries are likely overkill for basic metrics like response times or endpoint reachability as they are usually created when the goal is to make detailed performance metrics available.
- The applications require code changes for making use of the libraries. So if an application did not have the code before, it would need a revamp.

This is why blackbox exporters are commonly used in monitoring stacks. The Blackbox exporter is mainly used to measure response times & availability.

Blackbox installation

1. Add a blackbox user

```
sudo useradd --no-create-home blackbox
```

2. Download and extract the Blackbox binary:

```
wget  
https://github.com/prometheus/blackbox\_exporter/releases/download/v0.21.0-rc.0/blackbox\_exporter-0.21.0-rc.0.linux-amd64.tar.gz  
tar -xvf blackbox_exporter-0.21.0-rc.0.linux-amd64.tar.gz  
sudo mkdir /etc/blackbox
```

3. Copy files from the blackbox setup:

```
sudo cp blackbox_exporter-0.21.0-rc.0.linux-amd64/blackbox_exporter  
/usr/local/bin/  
  
sudo cp blackbox_exporter-0.21.0-rc.0.linux-amd64/blackbox.yml  
/etc/blackbox/
```

4. Adding content to blackbox's configuration file

```
sudo vim /etc/blackbox/blackbox.yml

---
modules:
  http_prometheus:
    prober: http
    timeout: 5s
    http:
      method: GET
      valid_http_versions: ["HTTP/1.1", "HTTP/2"]
      fail_if_ssl: false
      fail_if_not_ssl: false
```

5. Give the user 'blackbox' permission to the file used to run the blackbox binary.

```
sudo chown blackbox:blackbox /usr/local/bin/blackbox_exporter
sudo chown -R blackbox:blackbox /etc/blackbox/*
```

6. Add the blackbox startup configs in the service script.

This will let us start, stop, restart & check its status easily.

```
sudo vim /etc/systemd/system/blackbox.service
```

```
[Unit]
Description=Blackbox
Wants=network-online.target
After=network-online.target

[Service]
User=blackbox
Group=blackbox
Type=simple
ExecStart=/usr/local/bin/blackbox_exporter --
config.file=/etc/blackbox/blackbox.yml --web.listen-
address="0.0.0.0:9115"

[Install]
WantedBy=multi-user.target
```

7. Run the following to add the above service unit and start blackbox.

```
sudo systemctl daemon-reload
sudo systemctl enable blackbox
sudo systemctl start blackbox
sudo systemctl status blackbox
```

The installation is complete. Let's test the blackbox out.

Testing Blackbox

1. Check if the process is running or failing.

```
sudo systemctl status blackbox
```

```
ubuntu@ip-172-31-24-50:~$ sudo systemctl status blackbox
● blackbox.service - blackbox
   Loaded: loaded (/etc/systemd/system/blackbox.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2022-05-27 19:36:08 UTC; 2min 28s ago
     Main PID: 3311 (blackbox_exporter)
        Tasks: 6 (limit: 1145)
       Memory: 2.3M
      CGroup: /system.slice/blackbox.service
              └─3311 /usr/local/bin/blackbox_exporter --config.file=/etc/blackbox/blackbox.yml --web.li

May 27 19:36:08 ip-172-31-24-50 systemd[1]: Started blackbox.
May 27 19:36:08 ip-172-31-24-50 blackbox_exporter[3311]: ts=2022-05-27T19:36:08.633Z caller=main.go:25>
May 27 19:36:08 ip-172-31-24-50 blackbox_exporter[3311]: ts=2022-05-27T19:36:08.634Z caller=main.go:25>
May 27 19:36:08 ip-172-31-24-50 blackbox_exporter[3311]: ts=2022-05-27T19:36:08.634Z caller=main.go:26>
May 27 19:36:08 ip-172-31-24-50 blackbox_exporter[3311]: ts=2022-05-27T19:36:08.635Z caller=main.go:41>
```

2. Black occupies port 9115 here. Check response.

```
curl http://localhost:9115/metrics
```

```
ubuntu@ip-172-31-24-50:~$ curl -i http://localhost:9115/metrics
HTTP/1.1 200 OK
Content-Type: text/plain; version=0.0.4; charset=utf-8
Date: Fri, 27 May 2022 19:39:33 GMT
Transfer-Encoding: chunked

# HELP blackbox_exporter_build_info A metric with a constant '1' value labeled by version, revision, branch, and goversion from which blackbox_exporter was built.
# TYPE blackbox_exporter_build_info gauge
blackbox_exporter_build_info{branch="HEAD",goversion="go1.18.1",revision="6b655cdaf8727ff9bc301bfd5027c373b776085",version="0.21.0-rc.0"} 1
# HELP blackbox_exporter_config_last_reload_success_timestamp_seconds Timestamp of the last successful configuration reload.
# TYPE blackbox_exporter_config_last_reload_success_timestamp_seconds gauge
blackbox_exporter_config_last_reload_success_timestamp_seconds 1.6536801686349049e+09
# HELP blackbox_exporter_config_last_reload_successful Blackbox exporter config loaded successfully.
# TYPE blackbox_exporter_config_last_reload_successful gauge
blackbox_exporter_config_last_reload_successful 1
# HELP blackbox_module_unknown_total Count of unknown modules requested by probes
# TYPE blackbox_module_unknown_total counter
blackbox_module_unknown_total 0
```

Blackbox access

Let's open port 9115 to access the UI @

<http://<public-ip-address>:9115>

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

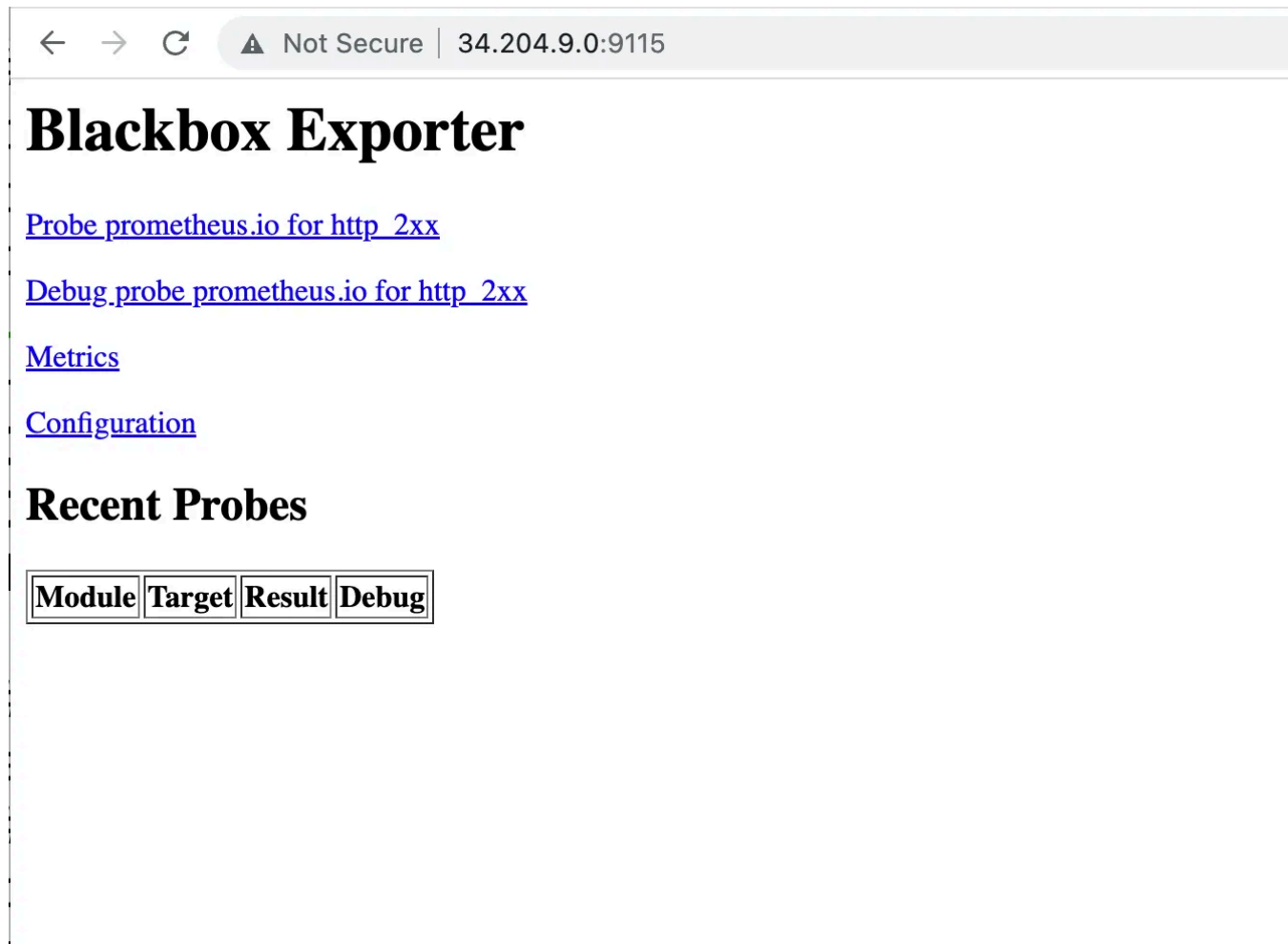
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-016ea551aa0d57d8e	SSH	TCP	22	Custom	Q 0.0.0.0/0 X	Delete
sgr-043013bdd85f8257a	Custom TCP	TCP	9090	Custom	Q 0.0.0.0/0 X	for prometheus Delete
-	Custom TCP	TCP	9115	Anywhere...	Q 0.0.0.0/0 X	for blackbox Delete

Add rule

Cancel

Preview changes

Save rules



So far, so good. Congratulations on installing Blackbox successfully.

Let's integrate blackbox into Prometheus now. We'll first understand important concepts to build a foundation on black-box and then configure a probe using a black-box exporter and send the probe metrics to Prometheus.

To setup Prometheus, follow this guide —

<https://medium.com/@shishirkh/monitoring-stack-setup-part-1-prometheus-grafana-372e5ae25402>

Configuring blackbox with Prometheus

In order to understand how blackbox integrates with Prometheus, it's important to understand some more concepts regarding blackbox first like:

- Modules
- Probing endpoints
- Relabelling

Modules

The blackbox exporter configuration file (/etc/blackbox/blackbox.yml) is made of **modules**. A module can be seen as one of the many probing configurations for the blackbox exporter.

Let's understand by taking certain practical examples.

Example 1

If I were to create configurations to probe an HTTP endpoint, I would have to create a module for it. This module would involve things like —

- The status codes should be considered valid. It's usually 200 but you can change it as well according to the use case.
- Basic auth configurations like username and password to reach the endpoint if the use case demands it.

Example 2

If I were to create a configuration to probe an HTTPS endpoint, I would have to create another module for it. This module would involve other things like:

- Configurations to check SSL validity since in this case, the probe should fail if the endpoint is not secure.

This is what 'modules' mean in brief. More details about these configurations can be found here —

https://github.com/prometheus/blackbox_exporter/blob/master/CONFIGURATION.md

Probing endpoints

Under blackbox exporters, there are two ways of querying-

1. Querying the exporter's metrics itself (itself means the blackbox exporter here). Usually available at `/metrics`.
2. Querying the exporter to scrape another target. Usually available at `/probe`.

It's the second one which is of more interest to us. To use the second type, we need to provide:

- Target: Target dictates 'where' the probing has to be applied. Target is the address of the endpoint which we want to probe.
- Module: This module would dictate how the probing would work. The module has to be defined in the exporter's configuration.

Example

The below curl probes the localhost:9090 target with `htt_prometheus` module specifications.

```
curl 'localhost:9115/probe?
target=localhost:9090&module=http_prometheus'
```

Since we have Prometheus running on 9090 and the 'http_prometheus' module defined earlier in /etc/blackbox/blackbox.yml — you can see from the response that the probe was successful.

Response to the above curl-

```
# HELP probe_success Displays whether or not the probe was a success
# TYPE probe_success gauge
probe_success 1
```

We are also getting many useful metrics, like latency by phase, status code, SSL status, or certificate expiry in Unix time among other things.

```
# HELP probe_http_duration_seconds Duration of http request by phase,
summed over all redirects
# TYPE probe_http_duration_seconds gauge
probe_http_duration_seconds{phase="connect"} 0.000512049
probe_http_duration_seconds{phase="processing"} 0.002300611
probe_http_duration_seconds{phase="resolve"} 7.3717e-05
```

```
probe_http_duration_seconds{phase="tls"} 0
probe_http_duration_seconds{phase="transfer"} 0.000107346

# HELP probe_http_status_code Response HTTP status code
# TYPE probe_http_status_code gauge
probe_http_status_code 200
```

Relabelling

When Prometheus scrapes a target, it assigns labels to the target — these labels help define the target such as its address, scrape_interval, etc.

For example

- A target's `job` label is set to the `job_name` value of the respective scrape configuration.
- The `__address__` label is set to the `<host>:<port>` address of the target.
- The `__scheme__` and `__metrics_path__` labels are set to the scheme and metrics path of the target respectively.
- The `__param_<name>` label is set to the value of the first passed URL parameter called `<name>`.

Relabeling is used to dynamically rewrite the label set on a target before it gets scraped. They are applied to the label set on each target in order of their appearance in the configuration file (/etc/prometheus/prometheus.yml)

Temporary labels help make relabeling easier. Let's see how.

Temporary labels

Labels starting with `__` will be removed from the label set by Prometheus after target relabeling is completed.

If a relabeling step needs to store a label value only temporarily (as the input to a subsequent relabeling step), use the `__tmp` label name prefix. This prefix is guaranteed to never be used by Prometheus itself.

`relabel_configs` section contains the new relabeling rules.

Let us understand with the help of an example.

```
relabel_configs:
  - source_labels: [__address__]
    target_label: __param_target
```


- source_labels: [__param_target]
target_label: instance
- target_label: __address__
replacement: localhost:9115

Let's see what is going on in this:

- First

- source_labels: [__address__]
target_label: __param_target

Here, we are taking the value from label '.__address__' and assigning it to '.__param_target' label.

- Second

- source_labels: [__param_target]
target_label: instance

Then we take the values from the label ‘__param_target’ and create a label ‘instance’ with those values.

- Third

```
- target_label: __address__  
  replacement: localhost:9115
```

Here, the value `localhost:9115` (the URI of our exporter) is assigned to label



🔍 Search

✍ Write



Prometheus integration

In order to instruct Prometheus to query “localhost:9115/probe?target=localhost:9090&module=http_prometheus” we need to add a configuration inside `prometheus.yml`.

But before, that — try to think of what this query is going to return for us. Actually, this query is querying the black-box exporter to carry out probing according to ‘http_prometheus’ configurations on ‘localhost:9090’.

We define the endpoints which we want to probe using the blackbox exporter via the Prometheus configuration file —
(/etc/prometheus/prometheus.yml)

Example

```
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'blackbox'
    metrics_path: /probe
    params:
      module: [http_prometheus] //specify the module to be used here
    static_configs:
      - targets:
        - http://localhost:9115
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: localhost:9115
```

The way it works is that blackbox uses its own configuration file
(/etc/blackbox/blackbox.yml) to define the different modules and depends